

WEB APPLICATION SECURITY ASSESSMENT

PENETRATION TEST REPORT

Altoro Mutual Online Banking — testfire.net

2 Critical

3 High

6 Medium

2 Low

Target: <http://testfire.net> / <https://testfire.net> (65.61.137.117)

Assessment Type: Black-box Web Application Penetration Test

Standard Referenced: OWASP Top 10 (2021)

Assessment Window: 03-04 October 2026

Prepared By: Mahesh — Security Researcher (Portfolio Engagement)

Classification: TLP:AMBER — For Portfolio / Training Use Only

1. Executive Summary

Overall Application Risk Rating

Driven by the highest-severity confirmed finding (CVSS v3.1 Base Score)

9.8 / 10 **CRITICAL**

A black-box web application penetration test was conducted against the Altoro Mutual online banking demo application hosted at **testfire.net**. The objective was to identify and validate security weaknesses across the application's authentication, authorization, input validation, session management and transport-layer controls, and to map each confirmed weakness to the relevant **OWASP Top 10 (2021)** category.

Testing identified **13 distinct findings**, including **2 Critical** and **2 High** severity issues. The most significant findings were a classic **SQL Injection authentication bypass** on the login form (`admin' or '1'='1`), which granted full administrative access to the online banking dashboard without valid credentials, and an **Insecure Direct Object Reference (IDOR)** in the fund transfer function that allowed an authenticated user to move an arbitrary amount of money into an account they do not own, by tampering with the `toAccount` and `transferAmount` parameters client-side.

Additional issues included a reflected Cross-Site Scripting (XSS) vulnerability in the search function, absence of account lockout / rate-limiting that permitted automated credential brute-forcing via Burp Intruder, verbose server error messages that leaked internal Java class and server version information, missing clickjacking protections (`X-Frame-Options`), weak TLS configuration (TLS 1.0/1.1 still enabled, 1024-bit Diffie-Hellman parameters), and transmission of login credentials over unencrypted HTTP.

Collectively, these findings indicate that the application — by design, as an intentionally vulnerable training target — fails to implement baseline controls across nearly every category of the OWASP Top 10. The recommendations in Section 9 describe the standard remediation for each class of issue as they would apply to a production banking application.

2. Scope & Engagement Details

Item	Detail
Target Host	testfire.net (resolved to 65.61.137.117)
In-Scope Subdomains	www, demo, demo2, altoro, ftp, localhost, evil (enumerated via <code>subfinder</code>)
In-Scope Ports / Services	80/tcp (HTTP), 443/tcp (HTTPS), 8080/tcp (HTTP-alt) — Apache Tomcat / Coyote JSP Engine 1.1
Test Type	Unauthenticated & authenticated black-box web application testing
Access Level	No credentials supplied by target owner (public demo account testing only)
Testing Platform	Kali Linux (desktop) + Android mobile browser for cross-device verification
Standard Referenced	OWASP Top 10:2021, OWASP Web Security Testing Guide (WSTG)
Exclusions	Physical security, social engineering, denial-of-service testing

2.1 Detailed Testing Timeline

Testing was conducted across two sessions, reconstructed below from tool and application response timestamps captured during the engagement (times approximate, IST unless noted):

Date	Time Window (IST)	Activity
Fri, 03 Oct 2026	~14:20 – 15:00	Initial Burp Proxy interception of login flow; manual SQL injection probing on <code>uid</code> parameter; first authentication-bypass confirmed (<code>admin'--</code>).
Fri, 03 Oct 2026	~15:45 – 16:20	Reconnaissance phase: <code>dig</code> , <code>WHOIS</code> , <code>nmap -sV</code> , <code>sslsca</code> , <code>wafw00f</code> , <code>subfinder</code> , <code>whatweb</code> executed against testfire.net.
Fri, 03 Oct 2026	~19:59 (post-session review)	Evidence capture / screenshot review of initial findings.
Sat, 04 Oct 2026	~11:26 – 11:40	Mobile cross-device verification of SQLi bypass and fund-transfer flow on Altoro Mutual mobile web view.
Sat, 04 Oct 2026	~13:03 – 13:12	<code>sqlmap</code> automated confirmation of the <code>uid</code> injection point (<code>--level=5 --risk=3</code> , DB2 heuristic).
Sat, 04 Oct 2026	~13:45 – 14:55	Burp Intruder cluster-bomb credential testing (49 requests) against <code>/doLogin</code> ; reflected XSS testing on search; clickjacking harness build & validation.
Sat, 04 Oct 2026	~14:55 – 15:50	Fund-transfer business-logic testing: parameter tampering on <code>/bank/doTransfer</code> (IDOR), malformed-input error disclosure (<code>transferAmount</code> crash).

2.2 Security Controls Checklist

Beyond the findings in Section 8, the following baseline controls were explicitly reviewed. Items are reported honestly whether they passed, failed, or were outside the achievable scope of this black-box engagement — in line with responsible reporting practice.

Control	Result	Notes
Session cookie <code>HttpOnly</code> flag	Pass	Confirmed present on <code>JSESSIONID</code> via WhatWeb — mitigates basic cookie theft via JavaScript.
Session cookie <code>Secure</code> flag	Fail	Not applicable/enforced — primary site served over HTTP (see Finding #4).
CSRF token on state-changing requests	Fail	Confirmed absent on <code>/bank/doTransfer</code> — see Finding #12.
Account lockout / rate limiting	Fail	Confirmed absent — see Finding #3.
Multi-factor authentication	Fail	Single-factor (username/password) authentication only, observed throughout testing.
Transaction velocity / amount limits	Fail	No limit enforced — see Finding #11.
Password complexity policy	Not Tested	No self-registration or password-change page was in scope/discoverable during black-box testing; recommended for a follow-on authenticated assessment.
Session idle/absolute timeout	Not Tested	Requires extended session-monitoring over time; out of scope for this engagement window. Recommended for follow-on testing.
Server-side logging / alerting on abuse	Inferred Fail	No blocking, CAPTCHA, or visible defensive response observed across 49 Intruder requests or ~100+ <code>sqlmap</code> requests — see Finding #13.

3. Methodology

Testing followed a standard black-box methodology aligned to the OWASP Web Security Testing Guide:

Phase	Activity	Description
1	Reconnaissance	DNS resolution, WHOIS lookup, subdomain enumeration, port/service fingerprinting, WAF detection, TLS/cipher assessment.
2	Mapping	Manual crawl of the application (login, search, account summary, fund transfer, news/content pages) via Burp Suite Proxy.
3	Vulnerability Identification	Manual testing of authentication, input validation, session handling, transport security and business-logic flows.
4	Exploitation / Validation	Proof-of-concept exploitation using Burp Repeater/Intruder, sqlmap and a local clickjacking harness to confirm real-world impact.
5	Reporting	Findings mapped to OWASP Top 10:2021 categories with evidence, CVSS-style severity and remediation guidance.

3.1 Tools Used

Tool	Purpose
Nmap	Port scanning and service/version detection
WHOIS / dig	Domain registration and DNS resolution checks
Subfinder	Passive subdomain enumeration
WhatWeb	Web technology / cookie flag fingerprinting
WAFW00F	Web Application Firewall detection
ssllscan	TLS/SSL protocol & cipher suite assessment
Burp Suite Community (Proxy, Repeater, Intruder)	Traffic interception, manual request tampering, authentication & brute-force testing
sqlmap	Automated SQL injection detection against the login parameter
Custom HTML PoC	Clickjacking proof-of-concept harness (iframe embedding)

4. Reconnaissance & Information Gathering

Initial reconnaissance established the target's attack surface prior to active testing.

```
(mahesh@kali) ~  
$ nmap -sV testfire.net  
  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-03 16:20 +0530  
Nmap scan report for testfire.net (65.61.137.117)  
Host is up (0.024s latency).  
Other addresses for testfire.net (not scanned): 64:ff9b::413d:8975  
Not shown: 994 filtered tcp ports (no-response)  
PORT      STATE SERVICE      VERSION  
80/tcp    open  http         Apache Tomcat/Coyote JSP engine 1.1  
113/tcp   closed ident  
443/tcp   open  ssl/http     Apache Tomcat/Coyote JSP engine 1.1  
2000/tcp  open  cisco-scp?     
8080/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1  
8443/tcp  closed https-alt  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 224.08 seconds
```

Fig 4.1 — Nmap service scan: ports 80, 443, 8080 open running Apache Tomcat/Coyote JSP Engine 1.1

```
subfinder  
projectdiscovery.io  
  
[INF] Current subfinder version v2.16.0 (latest)  
[INF] Loading provider config from /home/mahesh/.config/subfinder/provider-config.yaml  
[INF] Enumerating subdomains for testfire.net  
www.testfire.net  
localhost.testfire.net  
altoro.testfire.net  
demo.testfire.net  
demo2.testfire.net  
evil.testfire.net  
ftp.testfire.net  
[INF] Found 7 subdomains for testfire.net in 32 seconds 803 milliseconds
```

Fig 4.2 — Subfinder: 7 subdomains enumerated (www, demo, demo2, altoro, ftp, evil, localhost)

```
Domain Name: testfire.net  
Registry Domain ID: 8303973_DOMAIN_NET-VDSN  
Registrar WHOIS Server: whois.registrar.amazon  
Registrar URL:   
Updated Date: 2026-06-10T13:57:11Z  
Creation Date: 1999-07-23T13:52:32Z  
Registrar Registration Expiration Date: 2027-07-23T13:52:32Z  
Registrar: Amazon Registrar, Inc.  
Registrar IANA ID: 468  
Registrar Abuse Contact Email: trustandsafety@support.aws.com  
Registrar Abuse Contact Phone: +1.202.422.2533  
Domain Status: clientTransferProhibited https://icann.org/epp/clientTransferProhibited  
Registry Registrant ID: Not Available From Registry  
Registrant Name: On behalf of testfire.net OWNER  
Registrant Organization: c/o whoisproxy.com  
Registrant Street: 684 Cameron Street  
Registrant City: Alexandria  
Registrant State/Province: VA  
Registrant Postal Code: 22314  
Registrant Country: US  
Registrant Phone: +1.483.19528  
Registrant Phone Ext:   
Registrant Fax:   
Registrant Fax Ext:   
Registrant Email: a734ef1e832c48cf8eeaa547eb95383d786a9d21a5ab33bf688001a936b9c3@testfire.net.whoisproxy.org  
Registry Admin ID: Not Available From Registry
```

Fig 4.3 — WHOIS: domain registrar, creation/expiration dates and nameservers

```
(mahesh@kali) ~  
$ dig testfire.net  
  
;<> DIG 9.20.23-1-Debian <>> testfire.net  
;; global options: +cmd  
;; Got answer:  
;;->HEADER=< opcode: QUERY, status: NOERROR, id: 1905  
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0  
  
;; QUESTION SECTION:  
;testfire.net. IN A  
  
;; ANSWER SECTION:  
;testfire.net. 86897 IN A 65.61.137.117  
  
;; Query time: 8 msec  
;; SERVER: 192.168.43.1853(192.168.43.1) (UDP)  
;; WHEN: Sat Oct 03 15:54:43 IST 2026  
;; MSG SIZE rcvd: 46
```

Fig 4.4 — dig A-record resolution to 65.61.137.117

```
(mahesh@kali) ~  
$ sslscan testfire.net  
Version: 2.1.5  
OpenSSL 3.6.2 7 Apr 2026  
  
Connected to 65.61.137.117  
  
Testing SSL server testfire.net on port 443 using SNI name testfire.net  
  
SSL/TLS Protocols:  
SSLV2 disabled  
SSLV3 disabled  
TLSv1.0 enabled  
TLSv1.1 enabled  
TLSv1.2 enabled  
TLSv1.3 disabled  
  
TLS Fallback SCSV:  
Server does not support TLS Fallback SCSV  
  
TLS renegotiation:  
Secure session renegotiation supported
```

Fig 4.5 — sslscan: TLS 1.0 and 1.1 still enabled; TLS 1.3 disabled; no Fallback SCSV

```
(mahesh@kali) ~  
$ wafw00f testfire.net  
  
WOOF  
404 Hack Not Found  
405 Not Allowed  
405 Forbidden  
502 Bad Gateway  
500 Internal Error  
  
- WAFW00F: v2.4.2 -  
The Web Application Firewall Fingerprinting Toolkit  
  
[*] Checking https://testfire.net  
[*] Generic Detection results:  
[-] No WAF detected by the generic detection  
[-] Number of requests: 7
```

Fig 4.6 — WAFW00F: no Web Application Firewall detected on testfire.net

The recon phase confirmed an outdated Apache Tomcat/Coyote stack, legacy TLS protocol support, no WAF in front of the application, and a broader subdomain footprint than the primary host — all of which expand the effective attack surface and are referenced in the findings below.

5. Summary of Findings — OWASP Top 10:2021 Mapping

#	Finding	OWASP Category	Severity	CVSS 3.1
1	SQL Injection — Authentication Bypass on Login	A03:2021 – Injection	Critical	9.8
2	Broken Access Control — IDOR in Fund Transfer (arbitrary account / amount)	A01:2021 – Broken Access Control	Critical	8.1*
3	No Account Lockout / Brute-Force Protection + Username Enumeration	A07:2021 – Identification & Authentication Failures	High	7.5
4	Cleartext Transmission of Login Credentials (HTTP)	A02:2021 – Cryptographic Failures	High	7.4
5	Reflected Cross-Site Scripting (XSS) in Search	A03:2021 – Injection	Medium	6.1
6	Verbose Server Error / Stack Trace Disclosure	A05:2021 – Security Misconfiguration	Medium	5.3
7	Clickjacking — Missing X-Frame-Options / CSP frame-ancestors	A05:2021 – Security Misconfiguration	Medium	4.3
8	Weak TLS Configuration (TLS 1.0/1.1, 1024-bit DHE)	A02:2021 – Cryptographic Failures	Medium	5.9
9	Outdated / Vulnerable Components (Apache Tomcat/Coyote legacy build)	A06:2021 – Vulnerable & Outdated Components	Low	5.0
10	Information Disclosure — Server banners, open-source notice, subdomain exposure	A05:2021 – Security Misconfiguration	Low	3.1
11	Insecure Design — No Transaction Limits or Velocity Controls	A04:2021 – Insecure Design	High	7.1
12	Software & Data Integrity Failures — Missing Anti-CSRF Protection	A08:2021 – Software & Data Integrity Failures	Medium	6.5
13	Security Logging & Monitoring Failures	A09:2021 – Security Logging & Monitoring Failures	Medium	N/A†

* Finding #2's CVSS Base Score (8.1) reflects technical impact only; business risk is rated Critical given direct, unbounded financial fraud capability in a banking application. † CVSS does not directly score detective/monitoring control gaps; Finding #13 is rated qualitatively. All scores calculated per the FIRST.org CVSS v3.1 specification; vector strings are listed with each finding in Section 8.

Not applicable to this engagement: **A10:2021 (Server-Side Request Forgery)** — the application exposes no functionality that fetches a remote resource based on user-supplied input, so SSRF could not be tested from a black-box perspective. All other nine OWASP Top 10:2021 categories were actively tested; eight (A01–A09, excluding A10) resulted in one or more confirmed findings.

6. Risk Matrix

Severity was assigned qualitatively based on exploitability and business impact (likelihood × impact):

Likelihood \ Impact	Low	Medium	High
High	Medium	High	Critical (#1, #2)
Medium	Low	Medium (#5,#6,#7,#8,#12,#13)	High (#3, #4, #11)
Low	Low (#9, #10)	Low	Medium

7. Attack Narrative — Exploitation Kill Chain

Individually, several findings in this report are moderate. Chained together, they form a complete, low-skill attack path from an anonymous internet connection to unauthorised fund movement inside a banking application, with no detection at any step. The sequence actually demonstrated during this engagement is reconstructed below.

Step	Action	Finding Used	Attacker State After Step
1	Passive & active recon — <code>subfinder</code> , <code>nmap</code> , <code>whatweb</code> , <code>wafw00f</code>	Recon (§4)	Confirms Apache Tomcat backend, no WAF in front of the app, 7 subdomains, legacy TLS — attack surface mapped with zero risk of detection.
2	Inject <code>admin'--</code> into the login <code>uid</code> field	#1 SQL Injection	Authenticates as an administrative user with no valid credentials ; lands on <code>/bank/main.jsp</code> .
3	Browse authenticated dashboard	(session from Step 2)	Reaches "Hello Admin User" — account summary, transaction history and Transfer Funds are now reachable.
4	Intercept the Transfer Funds request in Burp Proxy and tamper <code>toAccount</code> / <code>transferAmount</code>	#2 IDOR + #11 Insecure Design	Moves 2,323,232 units into an account never selected in the UI — no ownership check, no amount ceiling, no step-up confirmation.
5	(Alternative/parallel path) Run Burp Intruder cluster-bomb against <code>/doLogin</code>	#3 No Lockout + #13 Logging Failure	Recovers a second valid account (<code>jsmith/demo1234</code>) purely from response-length analysis, with zero blocking across 49 attempts — a parallel route to Step 2 that doesn't even require the SQLi bug.
6	(Escalation path) Host the reflected XSS payload on an attacker page and lure a logged-in victim	#5 XSS + #12 Missing CSRF	Because no anti-CSRF token exists, the same forged-transfer request from Step 4 can be auto-submitted from the victim's own authenticated browser — removing the need to compromise credentials at all.
7	Submit a non-numeric <code>transferAmount</code> to force an error	#6 Verbose Errors	Full stack trace confirms exact Tomcat version and internal class layout — accelerates follow-on exploitation (e.g. known Tomcat CVEs) if the direct path above were ever blocked.

Bottom line: every step above was demonstrated with evidence in this report. An attacker does not need all of these bugs — *any one* of Steps 2, 5, or 6 independently leads to unauthorised fund movement within the same session. The application has no secondary control (transaction limits, MFA, CSRF tokens, or monitoring/alerting) to stop the chain once any single entry point is used.

8. Detailed Findings

8.1 SQL Injection — Authentication Bypass

A03:2021 - Injection

CVSS v3.1 Base Score: 9.8 (Critical) | AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Critical

Description

The login form at `/doLogin` does not sanitise or parameterise the `uid` POST parameter before it is used in the backend SQL/DB2 query. Submitting a classic tautology payload (`admin' or '1'='1`) as the username, with any password, bypasses authentication entirely and logs the tester in as an administrative user.

Proof of Concept

```
POST /doLogin HTTP/1.1
Host: testfire.net
Content-Type: application/x-www-form-urlencoded

uid=admin' -- &passw=pass&btnSubmit=Login

HTTP/1.1 302 Found
Location: /bank/main.jsp
```

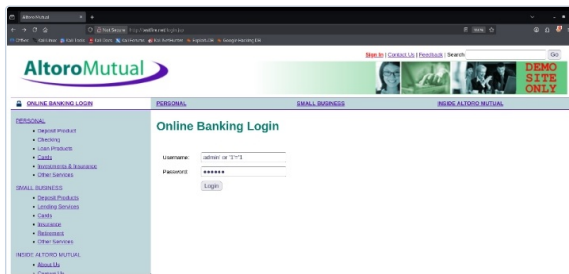


Fig 8.1a — Injection payload entered directly in the Username field

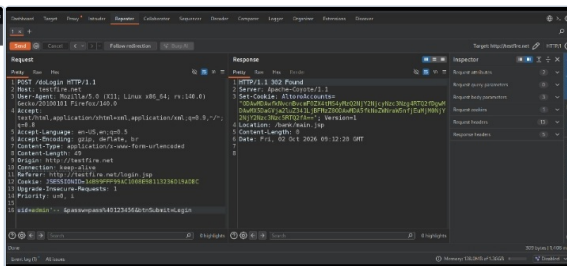


Fig 8.1b — Burp Repeater: 302 redirect to `/bank/main.jsp` confirms successful auth bypass

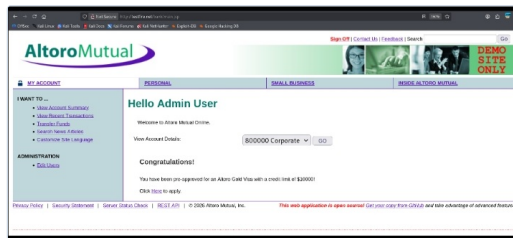


Fig 8.1c — "Hello Admin User" dashboard reached without valid credentials

Automated Validation (sqlmap)

```
sqlmap -u "http://testfire.net" --data="uid=test&passw=test" -p uid --dbms="IBM DB2" --level=5 --risk=3 --random-agent --dbs
```

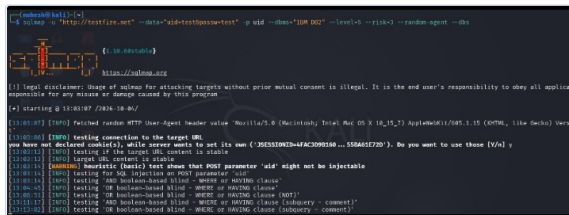


Fig 8.1d — sqlmap heuristic confirms POST parameter 'uid' is likely injectable

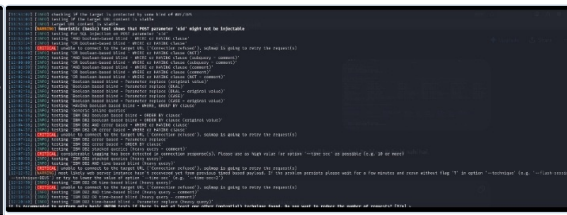


Fig 8.1e — sqlmap boolean/time-based blind test iterations against the login endpoint

Impact

Complete authentication bypass grants unauthorised access to any customer's banking dashboard and administrative functionality, including account summaries, recent transactions and the fund transfer feature (chained with Finding #2, this enables theft of funds from arbitrary accounts).

Remediation

Use parameterised queries / prepared statements (or an ORM) for all database access — never concatenate user input into SQL. Apply strict server-side input validation/allow-listing on the `uid` field, enforce least-privilege DB accounts, and deploy a WAF rule-set covering SQLi signatures as defence-in-depth.

8.2 Broken Access Control — IDOR in Fund Transfer

A01:2021 - Broken Access Control

CVSS v3.1 Base Score: 8.1 (High) | AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N

Business-impact escalated to **Critical** — enables direct, unlimited financial fraud in a banking context.

Critical

Description

The `/bank/doTransfer` endpoint accepts `fromAccount`, `toAccount` and `transferAmount` as client-controlled POST parameters with no server-side verification that the authenticated session actually owns `fromAccount`, and no upper bound / ownership check on `toAccount`. An attacker can intercept a legitimate transfer in Burp Proxy and alter the destination account and amount arbitrarily before forwarding the request.

Proof of Concept

```
POST /bank/doTransfer HTTP/1.1
Host: testfire.net
```

```
fromAccount=800002&toAccount=800018&transferAmount=2323232&transfer=Transfer+Money
```

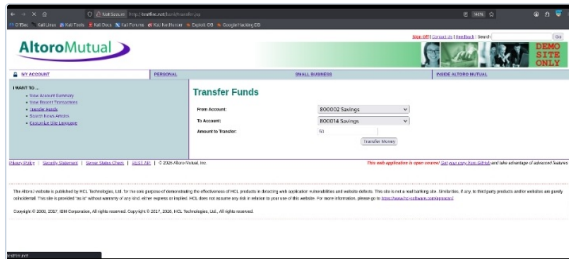


Fig 8.2a — Legitimate Transfer Funds form prior to tampering

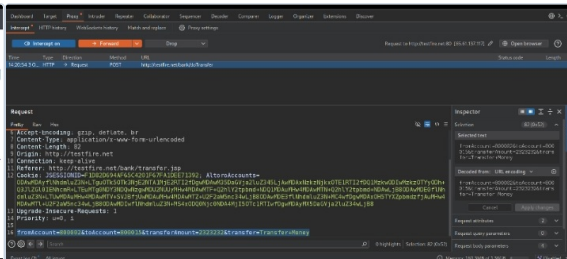


Fig 8.2b — Burp Proxy intercept: `toAccount` and `transferAmount` modified in-flight

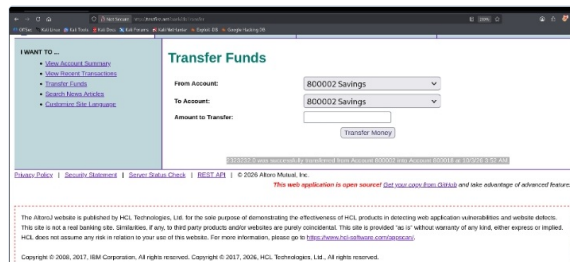


Fig 8.2c — Application confirms: "2323232.0 was successfully transferred from Account 800002 into Account 800018" — an account never selected by the legitimate user

Impact

A malicious or compromised session can move unlimited funds to any account number on the platform with no authorisation check, no transaction limit enforcement and no anti-CSRF token — a direct, high-impact fraud vector in a banking context.

Remediation

Enforce server-side ownership checks on `fromAccount` against the authenticated session on every request. Validate `toAccount` against the account holder's registered payees or require step-up verification for new payees. Add anti-CSRF tokens, server-side transaction limits, and out-of-band confirmation for high-value transfers.

8.3 No Account Lockout / Brute-Force Protection & Username Enumeration

A07:2021 - Identification & Authentication Failures

CVSS v3.1 Base Score: 7.5 (High) | AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

High

Description

The login endpoint imposes no rate-limiting, CAPTCHA or account lockout. A Burp Intruder *Cluster Bomb* attack was run against the `uid/passw` parameters using a small username and password wordlist; every attempt returned HTTP 302 with no throttling, and response-length differences between payload combinations allowed valid usernames to be distinguished from invalid ones (e.g. the `jsmith` / `demo1234` pair returned a markedly different response length than every failed combination, confirming a valid credential pair purely from response-size analysis).

Proof of Concept

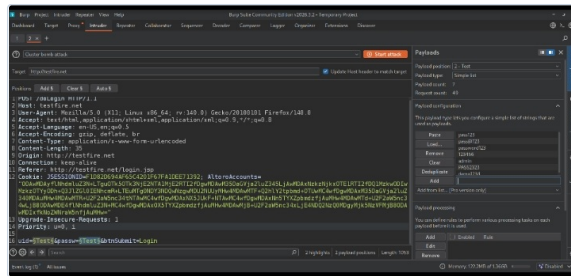


Fig 8.3a — Burp Intruder Cluster Bomb positions set on uid & passw

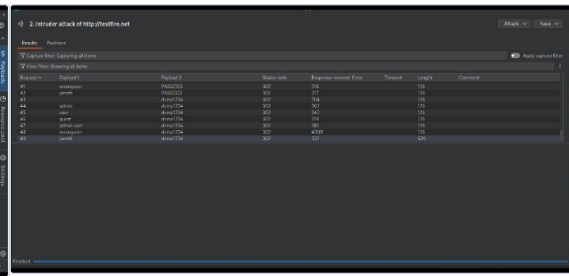


Fig 8.3b — Attack results: all 49 requests returned 302 with no lockout; response lengths vary by combination

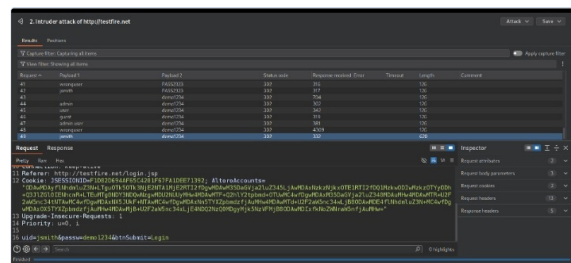


Fig 8.3c — Anomalous response length for jsmith/demo1234 flags a likely valid credential pair

Impact

Unlimited automated login attempts make credential-stuffing and brute-force attacks practical, and differing response characteristics enable username/credential enumeration without ever needing to guess a valid password outright.

Remediation

Implement progressive account lockout / exponential back-off after a small number of failed attempts, CAPTCHA after repeated failures, uniform error messages and uniform response timing/length regardless of whether the username exists, and multi-factor authentication for banking-grade applications.

8.4 Cleartext Transmission of Login Credentials

A02:2021 - Cryptographic Failures

CVSS v3.1 Base Score: 7.4 (High) | AV:A/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N

High

Description

The primary site (`http://testfire.net`) serves the login form and processes the `/doLogin` POST request over plain HTTP. Credentials, session cookies (`JSESSIONID`, `AltoroAccounts`) and all subsequent banking traffic are transmitted unencrypted and observable to any on-path attacker (e.g. on shared Wi-Fi).

Impact

Credentials and session tokens can be captured via network sniffing or ARP/DNS spoofing on the same network segment, leading to full account takeover without needing to exploit any application-layer bug.

Remediation

Enforce HTTPS site-wide with HSTS (`Strict-Transport-Security`), redirect all HTTP traffic to HTTPS, and mark all session cookies `Secure` and `HttpOnly`.

8.5 Reflected Cross-Site Scripting (XSS)

A03:2021 - Injection

CVSS v3.1 Base Score: 6.1 (Medium) | AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N

Medium

Description

The site search feature (`/search.jsp`) reflects the `query` parameter back into the page's HTML/attribute context without output encoding, allowing arbitrary HTML/JavaScript injection.

Proof of Concept

```
GET /search.jsp?query=0px;">XSS INJECTION</h1>
```

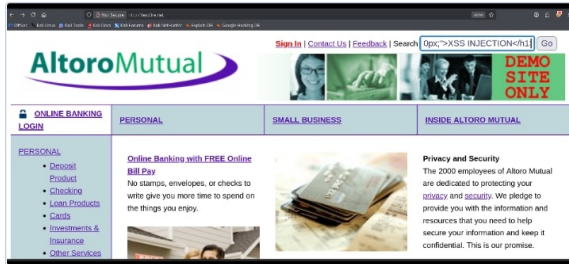


Fig 8.5a — Crafted payload injected into the search box

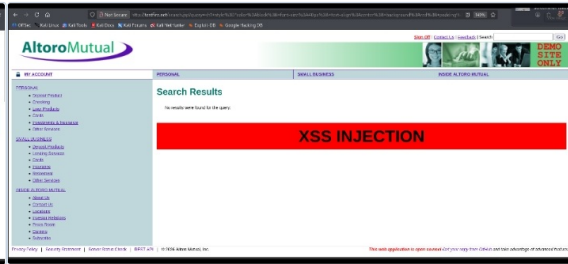


Fig 8.5b — Payload rendered unescaped on the results page ("XSS INJECTION" heading)

Impact

An attacker can craft a malicious link that, when clicked by a victim, executes arbitrary script in the victim's authenticated session — enabling session-cookie theft, phishing overlays, or forced actions (e.g. an auto-submitted fund transfer when chained with Finding #2).

Remediation

Context-aware output encoding on every reflected value (HTML entity encoding, JS string encoding, attribute encoding as appropriate), a strict Content-Security-Policy, and input validation/allow-listing for the search parameter.

8.6 Verbose Server Error / Stack Trace Disclosure

A05:2021 - Security Misconfiguration

CVSS v3.1 Base Score: 5.3 (Medium) | AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N

Medium

Description

Submitting a non-numeric value in the `transferAmount` parameter of `/bank/doTransfer` triggers an unhandled `NumberFormatException`, returning a full Java stack trace to the client.

Proof of Concept

```
fromAccount=800002&toAccount=800014&transferAmount=STRING-EXCEPTION-INJECTION-CRASH&transfer=Transfer+Money
```

HTTP Status 500 – Internal Server Error

```
java.lang.NumberFormatException: For input string: "STRING-EXCEPTION-INJECTION-CRASH"
    at com.ibm.security.appscan.altoromutual.servlet.TransferServlet.doPost(TransferServlet.java:69)
    ...
Apache Tomcat/7.0.92
```

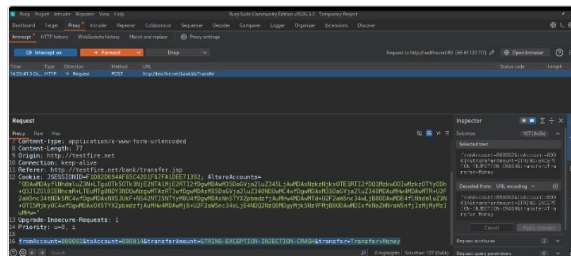


Fig 8.6a — Non-numeric value injected into transferAmount via Burp Proxy

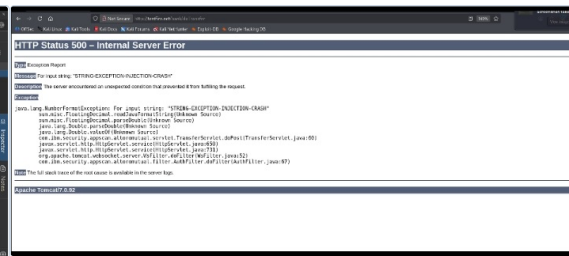


Fig 8.6b — Resulting HTTP 500 discloses full stack trace, class names and Tomcat version

Impact

Discloses internal package/class structure, the exact application server version (Apache Tomcat 7.0.92 — end-of-life), and confirms the backend is a Java servlet stack, which materially assists an attacker in selecting further exploits (e.g. known Tomcat CVEs).

Remediation

Implement global exception handling that returns a generic error page to the client, validate and type-check all numeric input server-side before use, disable `showServerInfo`/stack-trace output in production, and log detailed errors server-side only.

8.7 Clickjacking — Missing X-Frame-Options / CSP

A05:2021 - Security Misconfiguration

CVSS v3.1 Base Score: 4.3 (Medium) | AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N

Medium

Description

The application does not send an `X-Frame-Options` header or a `Content-Security-Policy: frame-ancestors` directive, allowing any third-party page to embed testfire.net inside an `<iframe>`.

Proof of Concept

```
<iframe src="http://testfire.net" style="width:800px;height:600px;border:1px solid black;"></iframe>
```

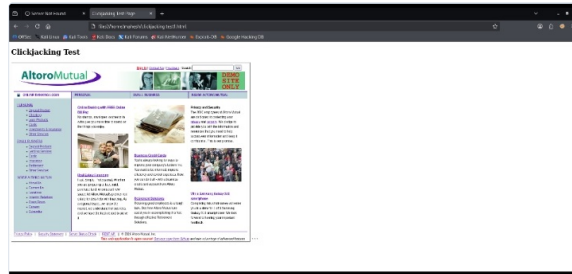


Fig 8.7 — Local clickjacking harness successfully embeds the live Altoro Mutual login page

Impact

Enables UI-redress ("clickjacking") attacks where an attacker overlays invisible/transparent controls over the framed banking page to trick an authenticated victim into performing unintended actions (e.g. a disguised "Transfer Money" click).

Remediation

Send `X-Frame-Options: DENY` (or `SAMEORIGIN`) and `Content-Security-Policy: frame-ancestors 'none'` on all authenticated pages.

8.8 Weak TLS Configuration

A02:2021 - Cryptographic Failures

CVSS v3.1 Base Score: 5.9 (Medium) | AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N

Medium

Description

`ssllscan` confirms the server still accepts the deprecated TLS 1.0 and TLS 1.1 protocols, does not support TLS 1.3, offers DHE cipher suites using only 1024-bit parameters (below the recommended 2048-bit minimum), and lacks TLS Fallback SCSV support.

```
(mahesh@kali)-[~]
$ ssllscan testfire.net
Version: 2.1.5
OpenSSL 3.0.2 7 Apr 2020
Connected to 65.01.137.117

Testing SSL server testfire.net on port 443 using SNI name testfire.net

SSL/TLS Protocols:
SSLV2      disabled
SSLV3      disabled
TLSv1.0    enabled
TLSv1.1    enabled
TLSv1.2    enabled
TLSv1.3    disabled

TLS Fallback SCSV:
Server does not support TLS Fallback SCSV

TLS renegotiation:
Secure session renegotiation supported
```

Fig 8.8a — TLS 1.0 / 1.1 enabled; TLS 1.3 disabled

```
TLS Compression:
Compression disabled

Heartbleed:
TLSv1.2 not vulnerable to heartbleed
TLSv1.1 not vulnerable to heartbleed
TLSv1.0 not vulnerable to heartbleed

Supported Server Cipher(s):
Preferred TLSv1.2 256 bits ECDHE-RSA-AES256-GCM-SHA384 Curve P-256 DHE 256
Accepted TLSv1.2 256 bits DHE-RSA-AES256-GCM-SHA384 DHE 1024 bits
Accepted TLSv1.2 128 bits ECDHE-RSA-AES128-GCM-SHA256 Curve P-256 DHE 256
Accepted TLSv1.2 128 bits DHE-RSA-AES128-GCM-SHA256 DHE 1024 bits
Accepted TLSv1.2 256 bits ECDHE-RSA-AES256-SHA384 Curve P-256 DHE 256
Accepted TLSv1.2 256 bits DHE-RSA-AES256-SHA256 DHE 1024 bits
Accepted TLSv1.2 128 bits ECDHE-RSA-AES128-SHA256 Curve P-256 DHE 256
Accepted TLSv1.2 128 bits DHE-RSA-AES128-SHA256 DHE 1024 bits
Accepted TLSv1.2 256 bits ECDHE-RSA-AES256-SHA Curve P-256 DHE 256
Accepted TLSv1.2 256 bits DHE-RSA-AES256-SHA DHE 1024 bits
Accepted TLSv1.2 128 bits ECDHE-RSA-AES128-SHA Curve P-256 DHE 256
Accepted TLSv1.2 128 bits DHE-RSA-AES128-SHA DHE 1024 bits
Preferred TLSv1.1 256 bits ECDHE-RSA-AES256-SHA Curve P-256 DHE 256
Accepted TLSv1.1 256 bits DHE-RSA-AES256-SHA DHE 1024 bits
Accepted TLSv1.1 128 bits ECDHE-RSA-AES128-SHA Curve P-256 DHE 256
```

Fig 8.8b — Cipher suites offering only 1024-bit DHE key exchange

Impact

Legacy protocol support exposes the application to downgrade attacks and known TLS 1.0/1.1 weaknesses; weak DHE parameters reduce the effective strength of the key exchange against a well-resourced attacker.

Remediation

Disable TLS 1.0 and 1.1, enable TLS 1.3, use 2048-bit (or ECDHE) key exchange only, and enable Fallback SCSV / strict cipher ordering.

8.9 Vulnerable & Outdated Components

A06:2021 - Vulnerable & Outdated Components

CVSS v3.1 Base Score: 5.0 (Medium) | AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L

Indicative base score; actual severity depends on the specific CVE exploited on this Tomcat build.

Low

Server banners and error pages confirm **Apache Tomcat/Coyote JSP Engine 1.1** and, during the error-disclosure test, specifically **Apache Tomcat 7.0.92** — an end-of-life major version with multiple publicly known CVEs. *Remediation:* upgrade to a current, supported Tomcat/Java EE server and maintain a patch cadence tied to vendor security advisories.

8.10 Information Disclosure (Banners, Subdomains, Source Notice)

A05:2021 - Security Misconfiguration

CVSS v3.1 Base Score: 3.1 (Low) | AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N

Low

whatweb confirms the `Server: Apache-Coyote/1.1` banner is exposed on every response; the footer discloses "This web application is open source! Get your copy from GitHub"; and passive subdomain enumeration (subfinder) revealed seven live or historical hostnames (including `demo2.testfire.net` and `evil.testfire.net`) that widen the reconnaissance surface available to an attacker. *Remediation:* suppress version banners, remove unnecessary disclosure text from production-facing pages, and inventory/retire unused subdomains.

8.11 Insecure Design — No Transaction Limits or Velocity Controls

A04:2021 - Insecure Design

High

CVSS v3.1 Base Score: 7.1 (High) | AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N

Description

This is a design-level root cause underlying Finding #2, called out separately because it would remain exploitable even after a narrow IDOR fix. The fund transfer workflow has **no maximum transaction amount, no daily/velocity limit, no step-up confirmation for high-value transfers, and no anomaly detection** for out-of-pattern activity. The same evidence used in Finding #2 (Fig 8.2c) demonstrates this directly: a single request moved **2,323,232** currency units — over two million — in one transaction, with the application raising no warning, confirmation step, or soft limit of any kind.

Impact

Even if parameter-level authorisation (Finding #2) is patched, the absence of transaction ceilings means a single compromised/authorised session (e.g. via phished credentials or session hijack) can drain an account in one request, with no secondary control to limit blast radius.

Remediation

Introduce server-side maximum-per-transaction and daily cumulative limits, step-up authentication (MFA / OTP) for transfers above a risk-based threshold, and real-time anomaly/fraud-scoring on transaction patterns — these are design controls that must exist independently of access-control fixes.

8.12 Software & Data Integrity Failures — Missing Anti-CSRF Protection

A08:2021 - Software & Data Integrity Failures

Medium

CVSS v3.1 Base Score: 6.5 (Medium) | AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N

Description

Every intercepted `/bank/doTransfer` and `/doLogin` request body captured during this engagement (see the raw request bodies reproduced in Findings #1 and #2) contains only functional parameters — `fromAccount`, `toAccount`, `transferAmount`, `transfer` / `uid`, `passw`, `btnSubmit` — with **no CSRF/anti-forgery token** in the body, headers, or as a required cookie-bound value. State only carries the ordinary `JSESSIONID` / `AltoraAccounts` session cookies.

Impact

Combined with the reflected XSS in Finding #5 or a simple external auto-submitting HTML form, an attacker can forge a cross-site request that executes a fund transfer (or other state-changing action) using a logged-in victim's browser session without their knowledge or consent — this is the classic CSRF chain and is directly enabled by the integrity gap observed here.

Remediation

Issue a unique, unpredictable, session-bound CSRF token on every state-changing form and validate it server-side on submission; additionally set `SameSite=Lax` or `Strict` on session cookies as defence-in-depth.

8.13 Security Logging & Monitoring Failures

A09:2021 - Security Logging & Monitoring Failures

Medium

CVSS v3.1 Base Score: N/A — Informational / Detective-Control Gap (CVSS does not score monitoring deficiencies directly)

Description

Across two high-volume automated attacks run during this engagement — a 49-request Burp Intruder cluster-bomb credential attack (Fig 8.3b) and a multi-stage `sqlmap` run with `--level=5 --risk=3` issuing well over a hundred boolean/time-based blind probes (Fig 8.1d-e) — the application returned normal `302`/error responses throughout with **no CAPTCHA challenge, no IP-based throttling, no account lockout, and no visible blocking page** at any point. This is consistent with an absence of real-time abuse detection or alerting on the authentication and query layers.

Impact

Without detection and alerting, the attacks demonstrated in this report (credential brute-forcing, SQL injection probing, IDOR exploitation) could be carried out against a production instance with no defender visibility until after funds or data had already been compromised, significantly increasing dwell time and impact.

Remediation

Implement centralised logging of authentication failures, anomalous parameter tampering, and high-value transactions; forward to a SIEM with alerting thresholds (e.g. N failed logins/minute, out-of-range transfer amounts); add WAF/IPS rules tuned to the injection and brute-force patterns demonstrated in this report.

9. Recommendations Summary

Priority	Recommendation	Addresses
1	Rewrite all database access using parameterised queries / prepared statements; never concatenate user input into SQL.	Finding #1
2	Add server-side ownership & authorization checks on every state-changing request (fund transfer, account actions); add anti-CSRF tokens and transaction limits.	Finding #2
3	Implement account lockout, rate-limiting, CAPTCHA and MFA on authentication endpoints; return uniform error responses.	Finding #3
4	Enforce HTTPS + HSTS site-wide; mark cookies Secure & HttpOnly.	Finding #4
5	Apply context-aware output encoding and a strict CSP to eliminate reflected/stored XSS.	Finding #5
6	Add global exception handling; disable verbose stack traces in production.	Finding #6
7	Send X-Frame-Options / CSP frame-ancestors on all pages.	Finding #7
8	Disable TLS 1.0/1.1, enable TLS 1.3, use 2048-bit+ / ECDHE key exchange only.	Finding #8
9	Patch/upgrade the application server and all third-party components on a regular cadence.	Finding #9
10	Suppress server/version banners; remove unnecessary disclosure text; inventory and retire unused subdomains.	Finding #10
11	Introduce per-transaction and daily transfer limits, step-up MFA/OTP for high-value or first-time-payee transfers, and real-time fraud/anomaly scoring.	Finding #11
12	Add session-bound anti-CSRF tokens to every state-changing request; set <code>SameSite=Lax/Strict</code> on session cookies.	Finding #12
13	Centralise authentication and transaction logging into a SIEM with alerting thresholds; add WAF/IPS tuned to the attack patterns in this report.	Finding #13

10. Conclusion

This assessment confirmed vulnerabilities spanning **Broken Access Control (A01)**, **Cryptographic Failures (A02)**, **Injection (A03)**, **Insecure Design (A04)**, **Security Misconfiguration (A05)**, **Vulnerable & Outdated Components (A06)**, **Identification & Authentication Failures (A07)**, **Software & Data Integrity Failures (A08)**, and **Security Logging & Monitoring Failures (A09)** — **eight of the ten** OWASP Top 10:2021 categories — against the Altoro Mutual demo banking application. The Attack Narrative in Section 7 shows these are not isolated issues: the SQL Injection authentication bypass, the IDOR-based fund transfer manipulation, the missing transaction limits, and the absent CSRF protection chain together into multiple independent paths from an anonymous connection to unauthorised fund movement, with no logging or monitoring control (A09) in place to detect any of them. Only **Server-Side Request Forgery (A10)** was not applicable to this application's functionality, and could not be tested from a black-box perspective.

As an intentionally vulnerable training platform, testfire.net served its purpose well: it allowed systematic, hands-on practice across reconnaissance, authentication testing, injection, business-logic abuse, and transport-layer assessment — all mapped cleanly to industry-standard OWASP categories for this portfolio engagement. The remediation guidance in Section 9 reflects the standard fix for each issue class as it would be applied to a production system.

Scope reminder: All testing in this report was performed against testfire.net, IBM/HCL's publicly provided intentionally-vulnerable training application. Findings, payloads and techniques documented here are intended for authorised security testing, training and portfolio demonstration only.